National University of Singapore

SCHOOL OF COMPUTING

CS2030 — PROGRAMMING METHODOLOGY II
(Semester 1: AY2025/2026)

Nov / Dec 2025 Time Allowed: 2 Hours

INSTRUCTIONS TO CANDIDATES

1. This assessment contains **FOURTEEN(14)** questions.
2. Answer **ALL** questions. The maximum mark is **40**.
3. This is an **EXAMPLIFY-SECURE OPEN-BOOK** assessment. You may refer to your lecture notes, recitation guides, lab codes, and the Java API.
4. You will be liable for disciplinary action which may result in expulsion from the University if you are found to have contravened any of the clauses below,
   - Violation of the NUS Code of Student Conduct (in particular the part on Academic, Professional and Personal Integrity), NUS IT Acceptable Use Policy or NUS Examination rules.
   - Possession of unauthorized materials/electronic devices.
   - Bringing your mobile phone or any storage/communication device with you to the washroom.
   - Unauthorized communication e.g. with another student.
   - Reproduction of any exam materials outside of the exam venue.
   - Photography or videography within the exam venue.
   - Plagiarism, giving or receiving unauthorised assistance in academic work, or other forms of academic dishonesty.
5. Once you have completed the assessment,
   - Click on the "Exam Controls" button and choose "Submit Exam".
   - Check off "I am ready to exit my exam" and click on "Exit" to upload your answers to the server.
   - You will see a green confirmation window on your screen when the upload is successful. Please keep this window on your screen.
   - If you do not see a green window, please disconnect and reconnect your WiFi and try again.
   - Please be reminded that it is your responsibility to ensure that you have uploaded your answers to the Software.

The following problem statement is for questions (1) to (5). You are advised to read through all the questions before attempting.

You are tasked to develop a delivery order tracker for a new local delivery service LaMovMov®. Tracking of each delivery order starts from a package that is shipped to Singapore, followed by a number of transits until it has been delivered.
A delivery order can be one of three stages:

- `Shipped` represents the starting stage of the delivery order with the package already shipped to the country. At this stage, the order can only be tagged for transit to another transit point;
- `InTransit` represents the stage of a package in transit. At this stage the order can either be tagged to another transit point, or marked as delivered;
- `Delivered` represents the final stage of the delivery. No further action can be taken at this stage.

For the purpose of tracking, a transaction record class `Record` has been designed for you with the following public methods. This is different from the Java keyword `record`.

- `public Record append(Record otherRecord)`

  Appends a record `otherRecord` to the existing record, and returns a new `Record`.

- `public String toString()`

  Returns a string representation of a record or appended records, e.g.

  - rec_1
  - rec_1 `->` rec_2
  - rec_1 `->` rec_2 `->` rec_3, etc.

  where rec_i is the string representation of the corresponding record i.

Examples of valid delivery tracking pipelines are given below. Assume that records `rec1`, `rec2`, ... have already been declared and initialized.

```
jshell> Shipped.of(rec1)
$.. ==> Shipped

jshell> Shipped.of(rec1).tagged(rec2)
$.. ==> InTransit

jshell> Shipped.of(rec1).tagged(rec2).tagged(rec3)
$.. ==> InTransit

jshell> Shipped.of(rec1).tagged(rec2).tagged(rec3).delivered(rec4)
$.. ==> Delivered
```

You are reminded to keep to the programming discipline instilled throughout the course.

1. **[3 marks]** Write an immutable `Order` class to support the three delivery order stages with the following constraints: .

   - do not allow objects of this class to be instantiated;
   - the only methods allowed are constructors

   The `Delivered` class has been provided for you.

```
class Delivered extends Order {
    Delivered(Order order, Record rec) {
        super(order, rec);
    }

    public String toString() {
        return "Delivered";
    }
}
```

   ANSWER:

2. [**4 marks**] Write the `InTransit` class. Sample delivery pipelines involving the `InTransit` stage are reproduced below:

```
jshell> Shipped.of(rec1).tagged(rec2)
$.. ==> InTransit

jshell> Shipped.of(rec1).tagged(rec2).tagged(rec3)
$.. ==> InTransit

jshell> Order order = Shipped.of(rec1).tagged(rec2).tagged(rec3)
order ==> InTransit

jshell> Shipped.of(rec1).tagged(rec2).tagged(rec3).delivered(rec4)
$.. ==> Delivered
```

You will be required to write the `Shipped` class in question (3).

ANSWER:

3. [**3 marks**] Write the `Shipped` class. Note that a delivery pipeline can only start with `Shipped.of(..)`. Sample delivery pipelines involving the `Shipped` stage are reproduced below:

```
jshell> Shipped.of(rec1)
$.. ==> Shipped

jshell> Order order = Shipped.of(rec1)
order ==> Shipped

jshell> Shipped.of(rec1).tagged(rec2)
$.. ==> InTransit

jshell> Shipped.of(rec1).delivered(rec4)
|  Error:
|  cannot find symbol
|    symbol:   method delivered(Record)
|  Shipped.of(rec1).delivered(rec4)
|  ^-----------------------^
```

Note from the above that the latter delivery pipeline is invalid.

ANSWER:

4. [**2 marks**] Thus far, we have yet to be able to observe the appended records. In this question, you are to suggest an appropriate `view` method in the `Order` class to support viewing of any stage during the process of delivery.

   You are free to design the `view` method in any way you wish, bearing in mind the "Tell-Don't-Ask" principle. You are not allowed to return a `Record` or even a `String`. You are also not allowed to introduce side-effects in the `Order` class, including any form of output.

   Write the method definition here; you do not need to modify the answer in question 1.

   ANSWER:

5. [**2 marks**] You are given the following `Stream` pipeline comprising several delivery orders of different order stages.

```
jshell> Stream<Order> orders = Stream.of(Shipped.of(rec1),
   ...>    Shipped.of(rec1).tagged(rec2),
   ...>    Shipped.of(rec1).tagged(rec2).tagged(rec3),
   ...>    Shipped.of(rec1).tagged(rec2).tagged(rec3).delivered(rec4))
orders ==> java.util.stream.ReferencePipeline$..
```

Complete the stream pipeline to show how the `view` method in question (4) can be used to output the following:

rec_1
rec_1 `->` rec_2
rec_1 `->` rec_2 `->` rec_3
rec_1 `->` rec_2 `->` rec_3 `->` rec_4

where rec_1, rec_2, ... are the string representations of the corresponding records `rec1`, `rec2`, ...

Your answer should begin with

```
orders.
```

ANSWER:

6. [**4 marks**] *This question is an extension of the preceding OOP design problem statement, and should be treated as a separate question.*

   You may have noticed that your solutions for the preceding questions may have allowed the following invalid instantations:

```
jshell> new Delivered(Shipped.of(rec1), rec2)
$.. ==> Delivered

jshell> new InTransit(new Delivered(Shipped.of(rec1), rec2), rec3)
$.. ==> InTransit
```

   Suggest how you can prevent invalid instantations by writing additional interface(s) and/or abstract class(es). Rewrite the `Delivered` class only. There is no need to write the other `Shipped` or `InTransit` classes.

   You need to ensure that there are no cyclic dependencies.

   ANSWER:

7. [**2 marks**] The following is an extract from the Java API for `CompletableFuture`'s `thenCombine` methods:

```
public <U,V> CompletableFuture<V> thenCombine(
    CompletionStage<? extends U> other,
    BiFunction<? super T,? super U,? extends V> fn)
```

```
Returns a new CompletionStage that, when this and the other given stage
both complete normally, is executed with the two results as arguments to
the supplied function.
```

Implement the `thenCombine` method using `thenCompose` and `thenApply`. You need only write the method body following the method header given above.

ANSWER:

8. [**4 marks**] The following is an extract from the Java API for `Stream`'s two argument `reduce` function:

   ```
   public T reduce(T identity, BinaryOperator<T> accumulator)

   Performs a reduction on the elements of this stream, using the provided
   identity value and an associative accumulation function, and returns the
   reduced value.
   ```

   Suppose you have been employed by Oracle to extend the API with another `reduce` function that takes in an identity value, followed by a *curried function* with the following method signature.

   ```
   public <U> U reduce(U identity, Function<U, Function<T, U>> accumulator)
   ```

   An example of concatenating three integer elements into a string using this new `reduce` method is given below:

   ```
   jshell> Function<String, Function<Integer, String>> f = x -> y -> x + y
   f ==> $Lambda..

   jshell> Stream.of(1,2,3).reduce("", f)
   $.. ==> "123"
   ```

   Write the `reduce` method. You need only write the method body following the method header given above. You are not allowed to use `Stream's` three argument `reduce`.

   ANSWER:

9. [**2 marks**] Using the curried `reduce` method defined in question (8), write another `reduce` method with the following method header:

```
public <U> U reduce(U identity, BiFunction<U, T, U> accumulator);
```

You need only write the method body following the method header given above.

ANSWER:

10. [**2 marks**] You are given a skeleton of a computation context `Either<L,R>` that can be used to support one of two types: a left side value of generic type `L`, or a right side value of generic type `R`.

```
import java.util.Maybe;
import java.util.function.Function;

record Pair<L,R>(L left, R right) {}

class Either<L,R> {
    private final Pair<Maybe<L>, Maybe<R>> pair;

    private Either(Maybe<L> left, Maybe<R> right) {
        this.pair = new Pair<>(left, right);
    }

    static <L,R> Either<L,R> left(L left) {
        return new Either<L,R>(Maybe.of(left), Maybe.empty());
    }

    static <L,R> Either<L,R> right(R right) {
        return new Either<L,R>(Maybe.empty(), Maybe.of(right));
    }
    ...
}
```

Write the `map` method that maps a given mapper function to the right side value if there is one. Otherwise the left value is propagated.

```
jshell> Either.<Integer,String>left(1).map(x -> x + 1)
$.. ==> Pair[t=Maybe[1], u=Maybe.empty]

jshell> Either.<Integer,String>right("one").map(x -> x + 1)
$.. ==> Pair[t=Maybe.empty, u=Maybe[one1]]

jshell> Either.<Integer,String>left(1).map(x -> x * 1)
|  Error:
|  bad operand types for binary operator '*'
|    first type:  java.lang.String
|    second type: int
|  Either.<Integer,String>left(1).map(x -> x * 1)
|                                        ^---^
```

The method should make use of bounded wildcards.

ANSWER:

11. [**4 marks**] This question follows from question 10.

    Write the `flatMap` method that flatmaps a given mapper function to the right side value if there is one. Otherwise the left value is propagated.

```
jshell> Function<String, Either<Integer,Integer>> f = s ->
   ...>    Either.right(s.length())
f ==> $Lambda..

jshell> Either.<Integer,String>left(1).flatMap(f)
$.. ==> Pair[left=Maybe[1], right=Maybe.empty]

jshell> Either.<Integer,String>right("one").flatMap(f)
$.. ==> Pair[left=Maybe.empty, right=Maybe[3]]

jshell> Function<String, Either<Integer,Integer>> e = s -> Either.left(0)
e ==> $Lambda..

jshell> Either.<Integer,String>right("one").flatMap(e)
$.. ==> Pair[left=Maybe[0], right=Maybe.empty]
```

    The method should make use of bounded wildcards.

    ANSWER:

12. [**2 marks**] We shall now attempt to re-implement the `Try` computation context using the `Either` context defined in questions 10 and 11.

    The following skeleton class is given.

```
class Try<T> {
    private final Either<Exception,T> either;

    private Try(Either<Exception, T> either) {
        this.either = either;
    }

    static <T> Try<T> of(Supplier<? extends T> sp) {
        try {
            return new Try<T>(Either.right(sp.get()));
        } catch (Exception e) {
            return new Try<T>(Either.left(e));
        }
    }
}
```

    Write the `map` method. The method should make use of bounded wildcards.

    ANSWER:

13. **[3 marks]** This question follows from question 12.

    Write the `flatMap` method. The method should make use of bounded wildcards.

    ANSWER:

14. **[3 marks]** Given stream elements x_1, x_2, x_3, x_4, x_5, ... where x_i are consecutive stream elements, a right-reduced subtraction is given by

$$x_1 - (x_2 - (x_3 - (x_4 - (x_5 - ..))))$$

However, there is no corresponding right reduction operation in Java Stream.

Using a `map` and the two argument `reduce` with an associative accumulator, complete the `rightSubtract` function body to return the right-reduced subtraction of the stream elements.

```
record IntPair(int fst, int snd) {}

int rightSubtract(Stream<Integer> stream) {
    return stream.
}
```

Here are some sample runs.

```
jshell> rightSubtract(Stream.of(1))
$.. ==> 1

jshell> rightSubtract(Stream.of(1,2))
$.. ==> -1

jshell> rightSubtract(Stream.of(1,2,3))
$.. ==> 2

jshell> rightSubtract(Stream.of(1,2,3,4))
$.. ==> -2

jshell> rightSubtract(Stream.of(1,2,3,4,5))
$.. ==> 3
```

Ensure that both *sequential* and *parallel* reductions will give the same outcome.

Hint: the solution is very similar to the `(res, x) -> 2 * res + x` *reduction that we discussed in class.*

ANSWER:

— END OF PAPER —